

SIH26249 — 04 Data Models & Logic

SIH 2026 — SIH26249

Air Power: Predictive Maintenance & Fleet Availability

Ministry of Defence — Defence Services Staff College

04 — Data Models & Logic

All data is synthetic. Every row carries a synthetic marker; every API response carries `"data_class": "synthetic"`.

4.1 Entities (SQLite)

aircraft — `tail_no` TEXT PK (synthetic, e.g. SYN-01 — never real-format tail numbers), `type` TEXT, `base` TEXT, `mission_priority` INT 1–3, `status` TEXT cached (MC/PMC/NMC).

component — `id` INT PK, `tail_no` FK, `comp_class` TEXT $\in$ {engine, avionics, hydraulics, airframe}, `serial_no` TEXT (synthetic), `installed_cycles` INT, `installed_hours` REAL.

sensor_reading — `id` INT PK, `component_id` FK, `cycle` INT, `op_setting_1..3` REAL, `sensor_1..12` REAL, `synthetic` INT = 1.

maintenance_record — `id` INT PK, `tail_no` FK, `component_id` FK nullable, `opened_at` / `closed_at` timestamps, `task_type` $\in$ {scheduled, unscheduled}, `crew_hours` REAL, `spare_used_id` FK nullable.

spare — `part_no` TEXT PK (synthetic), `comp_class` TEXT, `qty_on_hand` INT, `qty_due_in` INT, `lead_time_h` REAL, `expedited_lead_time_h` REAL.

crew — `id` INT PK, `name` TEXT (synthetic), `trade` TEXT, `shift` TEXT, `available_h_per_day` REAL.

sortie — `id` INT PK, `tail_no` TEXT nullable, `date` DATE, `mission_type` TEXT, `priority` INT 1–3, `required_status` $\in$ {FMC, PMC}.

prediction — `id` INT PK, `component_id` FK, `run_id` TEXT, `predicted_rul_h` REAL, `ci_low` REAL, `ci_high` REAL, `created_at` timestamp.

scenario — `id` INT PK, `name` TEXT, `base_run_id` TEXT, `lever_json` TEXT, `result_mc_rate` REAL, `created_at` timestamp.

audit_log — `id` INT PK, `ts` timestamp, `actor` TEXT, `action` TEXT, `entity` TEXT, `entity_id` TEXT, `detail_json` TEXT, `prev_hash` TEXT, `row_hash` TEXT (hash chain).

Fleet scale for demo: ~40 tails (trivial for SQLite).

4.2 Synthetic data generator spec

Per component: choose a failure mode; exponential degradation $\text{health}(t) = 1 - a \cdot \exp(b \cdot t)$ with randomized `a`, `b` per unit; sensors = `baseline(health) + op-condition offsets` (3 discrete operating settings, C-MAPSS-style) + Gaussian noise; 12 sensors + 3 op settings per reading. RUL label = $(\text{failure_cycle} - \text{current_cycle}) \times \text{hours_per_cycle}$. Include: **5% infant-mortality units**, **censored no-fault units**, and **maintenance events that reset the degradation curve**.

Spares/crew/sorties/records generated from consistent distributions (a part failure consumes a spare; a fix consumes crew-hours; sorties demand tails). **Seed stored in metrics.json**; generator determinism test: same seed → identical SHA-256 of the DB.

4.3 ML pipeline spec (RUL)

1. **Features:** per-sensor rolling mean/std/min/max/slope over 10- and 30-cycle windows + current cycle + cycles_since_overhaul + one-hots for op-setting and comp_class.
2. **Target:** RUL in flight hours, **capped at 125 h** (piecewise-linear trick — the model learns "healthy" as a flat 125, degradation as the slope).
3. **Model:** `HistGradientBoostingRegressor` with early stopping, one model per component class.
4. **Validation:** split **BY TAIL** (no leakage — a tail is wholly in train or wholly in test).
5. **Evaluation:** RMSE/MAE on held-out tails **vs naive mean-RUL baseline**; results printed by `eval.py` and stored in `metrics.json`. This printout is judge evidence.
6. **Prediction intervals:** quantile GBM, or $\pm 1.5 \times$ validation MAE (document whichever is used).
7. **Registry:** `model.pkl` + `metrics.json` per component class.

Explicitly out of scope: LSTM/Transformer RUL, survival analysis (future work).

4.4 Availability-computation logic (the aggregator)

Pure function: `fleet_state` → `{tail_states, mc_forecast[7], unfilled_sorties[7]}`.

Tail NMC on day d if ANY of: - R1: any critical component `predicted_rul_h < 24` (critical horizon; threshold in config) - R2: any component `predicted_rul_h < spare.lead_time_h` AND `qty_on_hand == 0` (supply grounding — NMCS) - R3: required fix `crew_hours > available trade crew-hours` in the window (capacity grounding — NMCM)

States: MC (fully mission capable), PMC (partially — restricted mission set), NMC (not mission capable), split NMCM/NMCS/NMCB by which rule(s) fired.

Fleet MC rate(d) = MC tails / total tails. Unfilled high-priority sorties per day = headline capability-gap number. **All thresholds live in one config file** (`config/thresholds.yaml`) so a judge can read them.

4.5 What-if engine logic

A lever is a **pure state patch on a cloned snapshot** — the base state is never mutated: -

`expedite_spares(part_no)` → `lead_time_h := expedited_lead_time_h` -

`reassign_crew(from_trade, to_trade, hours)` → move N available hours between trades -

`cannibalize(donor_tail, recipient_tail, part)` → donor goes NMC-down-for-parts, recipient receives the donor part's remaining RUL; **guardrails: donor must be lower mission priority AND already NMC** (unit-tested refusals)

Re-run the aggregator on the clone (**target < 200 ms**); return `{mc_rate_before, mc_rate_after, tails_changed, unfilled_sorties_before, unfilled_sorties_after}`; persist every run as a

scenario row. "Apply" copies the scenario's lever set to the working plan and writes an audit entry.

4.6 NMC-driver ranking logic

Per NMC tail, the **binding constraint** = whichever rule (R1/R2/R3) grounds it (first-fire order documented in config). Fleet score per driver = (tails grounded by this driver) × (avg mission priority of those tails) × (days grounded within the 7-day forecast). Present top-5 as "**fix X → recovers Y tails.**"

4.7 Cannibalization guardrails (logic, not just UI)

1. Donor priority must be **strictly lower** than recipient priority.
2. Donor must already be **NMC** (never take a part off a flyable tail).
3. CANN-rate counter: actions per 100 sorties tracked; warn at configurable threshold (doctrinal reference point: FY2000 USAF average 11.6 per GAO-01-693T/GAO-02-86 — cited as context, not a target).
4. Every approved action writes a **debt-ledger entry** (donor tail, part, recipient, repayment queue position); repayment = part arrives → reinstall → debt cleared.
5. Unit tests: refusal cases for (1) and (2) are mandatory P0 tests.